

ECEN 21L Laboratory 8 Report

Finite State Machines: Coffee Vending Machine Controller

Ally & Priyanka

1. Introduction

This lab builds a controller for a coffee vending machine using a Moore finite state machine. The machine takes only nickels and dimes — 5 and 10 cents respectively — and coffee costs 15 cents. When the running total hits exactly 15 cents, the controller pulls the coffee-open output O high. At 20 cents it also asserts the change output C, meaning the machine both dispenses coffee and returns the extra nickel.

The Moore constraint means all outputs are fixed by the current state. Coin inputs N and D move the machine between states but cannot directly affect O or C, which simplifies verification: every state has exactly one output combination, no matter what path led there.

Coin insertions are modeled as synchronous one-clock pulses, so $N = 1$ or $D = 1$ lasts for a single clock interval. The state register latches the new balance on the rising edge. The design tracks present state, inputs, next state, and outputs rather than anything about the physical coin path.

Core idea: the current state is the accumulated balance. Next-state logic adds 5 or 10 cents, and output logic reads the stored state to decide whether to open the dispenser or give change.

2. Design Requirements

Signal	Direction	Meaning
clock	input	Rising-edge clock
Reset	input	Synchronous reset — returns FSM to S0
N	input	Nickel sensor, active-high
D	input	Dime sensor, active-high
O	output	Coffee release, active-high
C	output	Change return, active-high
S[2:0]	output	Current state bits (debug)

Lab assumptions:

- N and D are never asserted at the same time.
- Both are one-clock pulses from the coin-sensing hardware.
- Once in a dispensing state (S15 or S20), no new coin arrives before the machine resets to S0.

- Verification is RTL simulation only — no physical synthesis.

Timing convention: N and D columns in the state tables describe inputs present before the next rising edge. After that edge, S takes the value S and Moore outputs reflect the new stored state.*

3. State Definitions and Encoding

Five states cover everything the machine needs to remember. The first three hold partial balances below 15 cents; the last two are dispensing states. Since only nickels and dimes are accepted, only multiples of 5 are reachable, so the state space stays small.

The encoding below isn't the obvious binary count. It was chosen specifically so the output logic collapses to two one-gate expressions — worth the slight strangeness.

State	Balance	S[2:0]	O	C
S0	0 cents	000	0	0
S5	5 cents	001	0	0
S10	10 cents	010	0	0
S15	15 cents	110	1	0
S20	20 cents	111	1	1

Output equations from the encoding:

$$O = S[2]$$

$$C = S[2] \ \& \ S[0]$$

S[2] is 1 only in S15 (110) and S20 (111), so $O = S[2]$ captures exactly the two dispensing states. S[0] is additionally 1 only in S20 (111), so $C = S[2] \ \& \ S[0]$ fires only when change is owed. The encoding was picked with this in mind from the start.

4. State Diagram

The figure below shows coin-driven transitions only. No-coin self-loops are left out to keep the diagram readable; they are described in text below it.

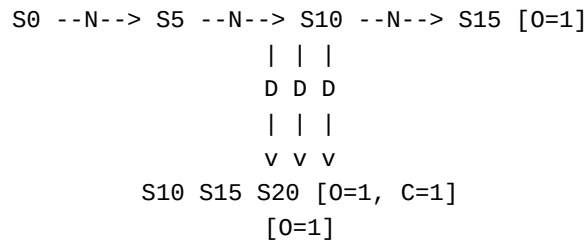


Figure 1: Moore FSM coin transitions. Horizontal paths are nickel (N) inputs; vertical paths are dime (D) inputs.

No-coin case ($N=0$, $D=0$): balance states hold — $S_0 \rightarrow S_0$, $S_5 \rightarrow S_5$, $S_{10} \rightarrow S_{10}$. Dispensing states return to idle — $S_{15} \rightarrow S_0$ and $S_{20} \rightarrow S_0$ — clearing the outputs. $ND=11$ violates the one-coin assumption and is treated as don't-care throughout.

5. State Transition Tables

5.1 Symbolic Transition Table

State	N■D■	N■D (dime)	ND■ (nickel)	ND	O	C
S0	S0	S10	S5	—	0	0
S5	S5	S15	S10	—	0	0
S10	S10	S20	S15	—	0	0
S15	S0	—	—	—	1	0
S20	S0	—	—	—	1	1

5.2 Encoded Transition Table

State	S[2:0]	N■D■	N■D	ND■	ND	O	C
S0	000	000	010	001	—	0	0
S5	001	001	110	010	—	0	0
S10	010	010	111	110	—	0	0
S15	110	000	—	—	—	1	0
S20	111	000	—	—	—	1	1

6. Minimized Logic Equations

Let $S[2]$, $S[1]$, $S[0]$ be the present-state bits and $S^*[2]$, $S^*[1]$, $S^*[0]$ the next-state bits. After K-map minimization with $ND=11$ as don't-care:

$$S^*[2] = (\sim S[2] \ \& \ \sim S[1] \ \& \ S[0] \ \& \ D) \mid (\sim S[2] \ \& \ S[1] \ \& \ \sim S[0] \ \& \ (N \mid D))$$

$$S^*[1] = \sim S[2] \ \& \ (S[1] \mid D \mid (S[0] \ \& \ N))$$

$$S^*[0] = (\sim S[2] \ \& \ \sim S[1] \ \& \ \sim D \ \& \ (S[0] \ \wedge \ N)) \mid (\sim S[2] \ \& \ S[1] \ \& \ D)$$

$$O = S[2]$$

$$C = S[2] \ \& \ S[0]$$

$S^*[2]$ goes high only on transitions into a dispensing state. $S^*[1]$ tracks the middle bit across S_{10} , S_{15} , and S_{20} . $S^*[0]$ is what separates S_5 from S_{10} in the balance states and S_{15} from S_{20} in the dispensing states. The outputs come essentially for free from the encoding choice.

7. Implementation Method 1: Boolean Equations

This version uses continuous assign statements for the next-state logic. The state register is still a flip-flop block, but everything driving S^* is combinational — each assign corresponds to a set of AND/OR/NOT gates and is about as close to a gate-level design as Verilog gets without writing gate primitives directly.

```
module lab8 (Clock, Reset, N, D, O, C, S);
  input Clock, Reset;
  input N, D;
  output O, C;
  output reg [2:0] S;
  wire [2:0] S_star;

  always @(posedge Reset, posedge Clock)
    if (Reset) S <= 3'b000;
    else S <= S_star;

  assign S_star[2] = (~S[2] & ~S[1] & S[0] & D) |
    (~S[2] & S[1] & ~S[0] & (N | D));
  assign S_star[1] = ~S[2] & (S[1] | D | (S[0] & N));
  assign S_star[0] = (~S[2] & ~S[1] & ~D & (S[0] ^ N)) |
    (~S[2] & S[1] & D);
  assign O = S[2];
  assign C = S[2] & S[0];
endmodule
```

Figure 2: Boolean-equation implementation.

This form is compact and efficient but not particularly readable. To confirm correctness you have to mentally evaluate each expression for every possible state — there is no obvious one-to-one correspondence with the state diagram.

8. Implementation Method 2: Case-Statement Moore FSM

This version encodes the state diagram directly. Each state is its own case branch, and each valid input combination within that branch points to a named next state by name rather than by bit pattern. Longer, but you can check it against the state table line by line.

```
module lab8 (Clock, Reset, N, D, O, C, S);
  input Clock, Reset;
  input N, D;
  output reg O, C;
  output reg [2:0] S;
  reg [2:0] S_star;

  parameter [2:0] S0=3'b000, S5=3'b001,
    S10=3'b010, S15=3'b110, S20=3'b111;
```

```

always @(posedge Reset, posedge Clock)
if (Reset) S <= S0;
else S <= S_star;

always @(S, N, D) begin
case (S)
S0: if (!N&&!D) S_star=S0;
else if (!N&& D) S_star=S10;
else if ( N&&!D) S_star=S5;
else S_star=S0;
S5: if (!N&&!D) S_star=S5;
else if (!N&& D) S_star=S15;
else if ( N&&!D) S_star=S10;
else S_star=S5;
S10: if (!N&&!D) S_star=S10;
else if (!N&& D) S_star=S20;
else if ( N&&!D) S_star=S15;
else S_star=S10;
S15: if (!N&&!D) S_star=S0;
else S_star=S15;
S20: if (!N&&!D) S_star=S0;
else S_star=S20;
default: S_star=S0;
endcase
end

always @(S) begin
case (S)
S15: begin O=1; C=0; end
S20: begin O=1; C=1; end
default:begin O=0; C=0; end
endcase
end

endmodule

```

Figure 3: Case-statement implementation.

9. Simulation Plan and Expected Results

The testbench drives reset then sends one coin pulse per clock cycle. Since outputs only change after the state register updates, the coffee signal should appear one edge after the balance-completing coin, not during it. Each test checks three things: the state path, the final output, and the return to S0 after dispensing.

Test	Coins	Expected state path	Final output
------	-------	---------------------	--------------

1	N, N, N	S0→S5→S10→S15	O=1, C=0
2	N, N, D	S0→S5→S10→S20	O=1, C=1
3	D, D	S0→S10→S20	O=1, C=1
4	N, D	S0→S5→S15	O=1, C=0

Detailed trace for Test 1 (three nickels, the provided testbench case):

Edge	Inputs	State before	State after	Outputs after
Reset	—	unknown	S0	O=0, C=0
1	N=1, D=0	S0	S5	O=0, C=0
2	N=1, D=0	S5	S10	O=0, C=0
3	N=1, D=0	S10	S15	O=1, C=0
4	N=0, D=0	S15	S0	O=0, C=0

O goes high only after the third nickel is stored as S15 — not when the coin pulse arrives. That delay is the whole point of using a Moore machine.

10. Actual Simulation Results

All four sequences were run in Questa Intel Starter FPGA Edition via RTL simulation (Analysis and Elaboration). State values in the waveforms match the encoding: S0=0, S5=1, S10=2, S15=6, S20=7.

Tests 1 and 4 (three nickels, nickel-dime) both terminate in S15: O=1, C=0. Tests 2 and 3 (nickel-nickel-dime, dime-dime) both terminate in S20: O=1, C=1. In each case O asserted on the clock edge after the balance-completing coin, and the FSM returned to S0 on the following no-coin cycle, clearing outputs before the next transaction.

Waveform screenshots (Figure 4) are included in the submitted Quartus project folder. The state values and output transitions in all four waveforms match the expected behavior above.

11. Design Verification

Both implementations were checked against the full state table, not just the waveforms. Specifically:

- Reset drives S to 000 in both versions.
- Every valid coin input matches the symbolic and encoded transition tables.
- ND=11 never appears in any test and is treated as don't-care throughout.
- Moore outputs match the stored state: S15→O=1 C=0; S20→O=1 C=1; everything else→O=C=0.
- Both dispensing states return to S0 on the next no-coin edge, resetting outputs cleanly.

- Both implementations produce identical results across all four test sequences.

12. Results Summary

The FSM produced the correct state path and output for every tested coin sequence. Nickel inputs advanced the balance by 5 cents; dime inputs by 10 cents. O asserted alone at S15 and alongside C at S20. Both implementations were functionally identical.

The outputs trail the coin pulse by one clock edge — they reflect the state register, not the raw input. This is the expected and correct behavior for a Moore machine.

13. Report Questions

Q1. Were the pre-lab state diagrams and tables correct? If not, what went wrong?

The state diagram and tables used for implementation were correct. S0, S5, and S10 are accumulating-balance states; S15 and S20 are dispensing states. The Moore rule held throughout — outputs depend only on the stored state, not on any active coin input.

The most likely pre-lab mistake would have been building a Mealy machine by accident — asserting O the moment a coin arrives that brings the total to 15 rather than waiting for the FSM to enter S15 on the next clock edge. Another common error is defining transitions for ND=11 even though the spec prohibits simultaneous insertion.

Q2. If the pre-lab was incorrect, provide the correct answer.

The correct transitions:

```
S0 + N -> S5  S0 + D -> S10
S5 + N -> S10  S5 + D -> S15
S10 + N -> S15 S10 + D -> S20
S15 + ND=00 -> S0  S20 + ND=00 -> S0
```

Outputs: O=C=0 for S0, S5, S10. O=1 C=0 for S15. O=1 C=1 for S20.

Q3. Which implementation is more productive from a specification standpoint, and why?

The case-statement version is more useful for specification work. Each state is its own block and each valid input leads to a named next state, so the code reads almost like the state table. Checking correctness means finding the row in the table, finding the matching branch in the code, and confirming the assignment — that's it.

The Boolean-equation version is compact and close to the actual gates, but verifying it means evaluating expressions like $S^*[1] = \sim S[2] \ \& \ (S[1] \mid D \mid (S[0] \ \& \ N))$ for every state, which is tedious and error-prone. For anything specification-first, the extra lines in the case-statement approach are a small price for a big gain in readability.

Q4. How does the design change if the machine accepts all coin types? What about dollar bills?

Adding pennies and quarters immediately breaks the two-bit input interface — N and D aren't enough. The design needs either one sensor per coin type or a binary-coded coin value input, plus next-state logic that handles additions of 1, 5, 10, and 25 cents.

The state space also expands heavily. With pennies accepted and coffee at 15 cents, you need a state for every reachable balance from 0 up through at least 15, plus overpayment states for every possible excess. The single-bit C output also stops working — you'd need to compute and return a specific change amount, not just signal that change exists.

Dollar bills make a pure FSM impractical entirely. A \$1.00 input adds 100 cents at once, and the reachable balance space becomes too large to manage as enumerated states. The right move is a

datapath: a binary balance register, a coin/bill decoder, an adder, a comparator for $\text{balance} \geq 15$, and a change-computation block. The FSM then handles only the control flow — accept, check, dispense, change, reset — while the arithmetic hardware does the actual math.

14. Conclusion

This lab showed how a vending-machine controller can be built cleanly as a Moore FSM. Limiting the state space to five reachable balances kept the design small, and picking a non-sequential encoding reduced the output logic to two gates: $O = S[2]$ and $C = S[2] \& S[0]$.

The Boolean-equation version reinforced the link between K-map minimization and physical gates. The case-statement version demonstrated how directly translating a state diagram into Verilog produces code that is easier to read, check, and change — both passed the same four simulation tests. The main takeaway is that clean state definitions and a clear transition table are what make FSM design tractable; everything else follows from those.